

Course Outline: Using Pre-Trained Models

Title: Pre-Trained Models: Find, Evaluate, and Use **Skill:** Skill x: Predictions Model **Learnpack:** + Cómo buscar y utilizar modelos ya creados **File:** s__-w16d48-como-buscar-y-utilizar-modelos-ya-creado **Prerequisite:** s__-w16d48-modelos-de-prediccion **Target Audience:** Beginner developers who understand what classifiers and regressors are and now want to use existing models without training from scratch **Total Lessons:** 8 **Format:** Mixed (12% hands-on = 1 lesson)

Course Description

You don't always need to train a model from scratch. For many tasks, a high-quality pre-trained model already exists — trained on millions of examples, validated, and ready to use. This course shows you where to find pre-trained models, how to evaluate whether one fits your task, and how to load it and run inference in Python using scikit-learn. The hands-on challenge puts all three steps together: find → load → predict.

Course Philosophy

This course emphasizes:

- The practical path to using ML: find an existing model before considering training one
 - Evaluation criteria for model selection: task fit, license, metrics, community support
 - The minimal code path to getting predictions from a pre-trained model
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Finding and Evaluating	2
02 - Using a Model	3
03 - Assessment	1
04 - Outro	1
Total	8

00.0 Pre-Trained Models

Learning Objectives:

- Define a pre-trained model as a model that has already been trained on a large dataset and is available for reuse

- Understand why using a pre-trained model is often the right first step
- Preview the find-evaluate-use workflow this course teaches

Content Outline:

- What a pre-trained model is: a model whose parameters were already optimized by training on a large dataset — you don't need to collect data, write training code, or wait for training to complete
- Why pre-trained models exist: training large models requires significant compute and data; researchers and companies train once and share the result; the community benefits from reusing their work
- The economics: a state-of-the-art image classifier trained on 1.2 million images for weeks is available for free in 5 lines of code
- When to use a pre-trained model vs. training from scratch:
 - Use pre-trained: your task is common (spam detection, sentiment analysis, image classification, named entity recognition); you have limited data; you need a baseline quickly
 - Train from scratch: your task is highly specialized; the domain is unusual; you have abundant proprietary data; privacy requires data not to leave your systems
- The three-step workflow this course teaches: (1) find a pre-trained model for your task; (2) evaluate whether it's a good fit; (3) load it and run inference
- Course structure: Section 01 covers finding and evaluating models; Section 02 covers using a model in code

Transitions: This lesson establishes the value of pre-trained models. Section 01 shows where to find them.

Key Principles:

- Most ML problems in production start with a pre-trained model, not with training from scratch
- A pre-trained model is a starting point — it can be further fine-tuned on your specific data if needed
- The "model card" (description, metrics, license, intended use) is the essential document for evaluating any pre-trained model

Examples:

- Using a pre-trained spam filter: `model.predict(email_text)` → "spam" — the model was trained on millions of emails; you use it in one line
- Using a pre-trained sentiment analyzer: `pipeline("sentiment-analysis")("I love this product")` → `{"label": "POSITIVE", "score": 0.99}` — takes 5 minutes to integrate
- Building from scratch: months of work, thousands of labeled examples, training infrastructure — justified only when pre-trained options don't exist

01.0 Finding Models

Learning Objectives:

- Name the major repositories where pre-trained models can be found
- Identify which repository is best for which type of task
- Describe what information to look for when browsing model repositories

Content Outline:

- **Major model repositories:**
 - **Hugging Face Hub (huggingface.co/models):** the largest community repository; over 500,000 models across NLP, computer vision, audio, and more; every model has a model card; searchable by task (text-classification, object-detection, etc.)
 - **scikit-learn built-in models:** pre-implemented algorithms (not pre-trained weights, but ready-to-use implementations); for tabular data; includes SVM, random forests, gradient boosting, etc.
 - **TensorFlow Hub (tfhub.dev):** pre-trained TensorFlow models for image, text, and video; strong in computer vision
 - **PyTorch Hub (pytorch.org/hub):** curated repository of PyTorch models; especially strong in research models
 - **Kaggle Models:** community-contributed models, often winners of competitions; real-world task focus
 - **ONNX Model Zoo (onnx.ai/models):** models in the ONNX format (interoperable across frameworks)
- **Browsing by task:** every major repository allows filtering by task type; start with the task you need (text-classification, image-classification, token-classification, etc.) and look at the most popular/downloaded models
- **What information to look for on a model's page:**
 - **Task:** what does the model predict? Does it match your need?
 - **Metrics:** accuracy, F1, BLEU, etc. — how well does it perform on benchmark datasets?
 - **Dataset:** what data was it trained on? Is it relevant to your domain?
 - **License:** can you use it in your project? (MIT, Apache 2.0 = permissive; CC-BY-NC = non-commercial only)
 - **Downloads/stars:** popularity as a proxy for reliability and community support
 - **Last updated:** is the model actively maintained?
- Section preview: 01.1 goes deeper into how to evaluate and choose between candidate models

Transitions: You know where to look. 01.1 covers how to evaluate the models you find.

Key Principles:

- Hugging Face Hub is the default starting point for NLP and increasingly for all modalities
- Always check the license before using a model in a production system
- Model popularity (downloads, stars) is a useful heuristic for reliability — heavily used models have more bug reports and community support

Examples:

- Finding a spam classifier: search Hugging Face for "text-classification spam" → find [jy46604790/Fake-News-Bert-Detect](https://huggingface.co/jy46604790/Fake-News-Bert-Detect) or similar → check metrics, license, and intended use

- Finding an image classifier: search for "image-classification" on Hugging Face → [google/vit-base-patch16-224](#) (trained on ImageNet) → check 224x224 input requirement
-

01.1 Evaluating a Model

Learning Objectives:

- Apply evaluation criteria (task fit, metrics, license, community support, model size) to choose between candidate models
- Read and interpret a model card to assess fitness for a given use case
- Recognize common red flags in model documentation

Content Outline:

- **The evaluation checklist:**
 1. **Task fit:** Does the model's stated task match your use case exactly? A model trained for "sentiment analysis on product reviews" may not work well on "sentiment analysis on medical notes"
 2. **Metrics on benchmark datasets:** benchmark scores tell you how well the model performs on standard test sets; e.g., a text classifier with 95% accuracy on SST-2 (movie review sentiment) is likely good for similar tasks
 3. **Training data relevance:** what domain/language/style was the training data in? English language models may perform poorly on Spanish text; models trained on formal text may fail on informal social media
 4. **License:** check before integrating into a product
 - MIT, Apache 2.0: permissive, use anywhere
 - CC-BY-NC: non-commercial only — can't use in a paid product
 - Proprietary: requires a commercial license
 5. **Model size and inference speed:** larger models (e.g., 7B parameter LLMs) produce better results but require more RAM and are slower; for edge or mobile deployment, smaller models are necessary
 6. **Community support and activity:** check GitHub stars, recent commits, open issues — an abandoned model may have known bugs
- **Reading a model card:**
 - The model card is the model's documentation — analogous to a README
 - It should describe: intended use, out-of-scope use, bias and fairness considerations, training data, evaluation metrics
 - A model card without bias/fairness information is a warning sign — the model may perform unevenly across demographic groups
- **Common red flags:**
 - No model card or very sparse documentation
 - Metrics only on training data (not a held-out test set) — the model may be overfitting
 - License is unclear or too restrictive for your use case
 - No mention of training data or domain
 - Very recent upload with zero downloads/reviews

Transitions: You can now find and evaluate models. Section 02 shows how to load and use them in code.

Key Principles:

- Always read the model card before using a model in production
- "Best benchmark score" $\neq$ "best for your task" — domain fit matters more than raw metrics
- License is a hard requirement, not a preference — check it first before evaluating anything else

Examples:

- Evaluating two spam classifiers:
 - Model A: 97% accuracy on standard benchmark, MIT license, trained on email data, 10,000 downloads ✓
 - Model B: 99% accuracy on social media text, CC-BY-NC license, 50 downloads — higher accuracy but wrong domain and restrictive license ✗
 - Red flag: "This model was trained on 1,000 examples from our proprietary dataset. It achieves 99% accuracy." — 99% on a tiny training set likely means overfitting
-

02.0 Using a Model

Learning Objectives:

- Describe the three steps for using a pre-trained model in code: load, prepare input, run inference
- Understand what "inference" means in the context of a pre-trained model
- Preview what the next two lessons cover

Content Outline:

- **What "using a model" means:**
 - Load: bring the model into memory — this is reading the model's learned parameters from a file or repository
 - Prepare input: transform your raw data into the format the model expects (features, normalization, tokenization, etc.)
 - Inference: pass the prepared input through the model to get a prediction — this is calling `model.predict()` or equivalent
- **The load-prepare-predict pipeline:**
 1. Install the required library (scikit-learn, transformers, tensorflow, etc.)
 2. Load the model (from a file, a library's built-in models, or a remote hub)
 3. Prepare the input in the format the model expects
 4. Call the predict function
 5. Interpret the output
- **What "inference" means:** the model is a function; inference is calling that function with new inputs to get predictions; the model's weights don't change during inference (unlike training)
- **Two main code paths this course shows:**
 - scikit-learn: `from sklearn.X import Y; model = Y(); model.fit(X, y); model.predict(X_new)` — used for tabular data

- Hugging Face `pipeline`: `from transformers import pipeline; classifier = pipeline("task"); classifier(text)` — used for text, images, audio
- Section preview: 02.1 covers the specific code for loading and running inference; 02.2 is the hands-on challenge

Transitions: The load-prepare-predict flow is the pattern. 02.1 shows the code.

Key Principles:

- Inference does not change the model — you are only reading from the model, not writing to it
- The biggest challenge in using a pre-trained model is preparing the input correctly — each model expects a specific format
- The output format also varies: some return class labels, some return probabilities, some return tensors — always check the model's documentation

Examples:

- scikit-learn flow: `iris = load_iris(); X, y = iris.data, iris.target; model = GaussianNB(); model.fit(X, y); model.predict([[5.1, 3.5, 1.4, 0.2]])`
 - Hugging Face flow: `classifier = pipeline("sentiment-analysis"); result = classifier("I love this course!")`
 - Input preparation matters: a sentiment model trained on English text receives "I love this" as a raw string; an image model expects a 224x224 RGB array
-

02.1 Loading and Running Inference

Learning Objectives:

- Load a pre-trained scikit-learn model and run predictions on new samples
- Interpret model output (class labels, class names) correctly
- Explain the difference between training the model and using it for inference

Content Outline:

- **Using scikit-learn with the iris dataset:**
 - The iris dataset is a canonical benchmark: 150 flower measurements (4 features: sepal length, sepal width, petal length, petal width) across 3 species (setosa, versicolor, virginica)
 - `load_iris()` loads the dataset; the features are already clean and normalized
 - GaussianNB (Gaussian Naive Bayes) is a simple classifier that trains in milliseconds on this dataset
 - The workflow: load dataset → fit model → predict on new samples → map prediction (0, 1, 2) to class names
 - This is equivalent to "loading a pre-trained model" in the sense that the model was already trained on the benchmark data — in production, you'd load from disk with joblib instead of retraining
- **The inference step:**
 - `model.predict(X_new)` returns an array of predicted class indices

- `model.predict_proba(X_new)` returns the probability for each class — useful for confidence-aware applications
- Mapping index to name: `iris.target_names[prediction]` converts 0 → "setosa", 1 → "versicolor", 2 → "virginica"
- **Loading from disk (the real pre-trained model pattern):**
 - Models are saved with joblib: `joblib.dump(model, 'iris_classifier.joblib')`
 - Loaded with: `model = joblib.load('iris_classifier.joblib')`
 - The loaded model is identical to the trained one — inference works the same way
- **Hugging Face pipelines (brief overview):**
 - `from transformers import pipeline; classifier = pipeline("text-classification")`
 - The pipeline downloads the model on first use, caches it, and wraps it with preprocessing
 - `classifier("The food was excellent")` → [{"label": "POSITIVE", "score": 0.99}]
 - Same load-prepare-predict pattern, just with a different tool

Transitions: You have the conceptual and practical knowledge. 02.2 is the hands-on challenge.

Key Principles:

- In production, you load a model once at startup, then run inference on every request — loading is the expensive step
- `predict()` returns class indices; always map to human-readable names for output
- `predict_proba()` gives you confidence — useful for deciding when to trust a prediction vs. defer to a human

Examples:

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

iris = load_iris()
model = GaussianNB()
model.fit(iris.data, iris.target)

# Inference on new samples
samples = [[5.1, 3.5, 1.4, 0.2], [6.7, 3.0, 5.2, 2.3]]
predictions = model.predict(samples)
probabilities = model.predict_proba(samples)

for i, (pred, proba) in enumerate(zip(predictions, probabilities)):
    print(f"Sample {i+1}: {iris.target_names[pred]} ({max(proba)*100:.1f}% confidence)")
```

Challenge Prompt: Using scikit-learn, load the iris dataset, train a GaussianNB classifier on the full dataset (treat this as loading a pre-trained model — the model is already known to work on this benchmark), and run inference on two new flower samples. Print the predicted species name for each sample.

Solution Code:

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

iris = load_iris()
X, y = iris.data, iris.target

model = GaussianNB()
model.fit(X, y)

new_samples = [
    [5.1, 3.5, 1.4, 0.2],
    [6.7, 3.0, 5.2, 2.3]
]

predictions = model.predict(new_samples)

for i, pred in enumerate(predictions):
    print(f"Sample {i+1}: {iris.target_names[pred]}")
```

Challenge Code:

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB

# Load the iris dataset into a variable called 'iris'
# your code here

# Extract features (X) and labels (y) from the dataset
# your code here

# Create a GaussianNB classifier and train it on the full dataset
# your code here

# Define two new flower samples to classify:
# Sample 1: sepal_length=5.1, sepal_width=3.5, petal_length=1.4,
# petal_width=0.2
# Sample 2: sepal_length=6.7, sepal_width=3.0, petal_length=5.2,
# petal_width=2.3
# your code here

# Run inference and store predictions
# your code here
```



```
# Print the predicted species name for each sample
# Expected output: "Sample 1: setosa" and "Sample 2: virginica"
# your code here
```

03.0 Assessment 🧠

Learning Objectives:

- Apply model selection criteria to scenarios
- Identify the correct steps in the load-prepare-predict workflow
- Recognize potential issues with pre-trained model usage

Question Format: Scenario-based (multiple-choice)

Questions:

1. You need to add spam detection to your email service. What should you do first?
 - A) Collect 10,000 spam/non-spam emails and train a model from scratch
 - B) Search Hugging Face for an existing text-classification model trained on email spam data ✓
 - C) Build a rule-based filter using keyword matching
 - D) Ask your users to label emails
2. A Hugging Face model card shows: "License: CC-BY-NC." Can you use this model in a paid SaaS product?
 - A) Yes — CC-BY-NC allows all uses as long as you credit the author
 - B) No — CC-BY-NC restricts commercial use ✓
 - C) Yes — licenses on Hugging Face don't apply to API usage
 - D) Yes — you can use any model for free if you don't redistribute it
3. You find two models for your task. Model A has 95% accuracy on a benchmark matching your domain; Model B has 99% accuracy but was trained on social media data and your task is formal legal text. Which should you start with?
 - A) Model B — higher accuracy always means better performance
 - B) Model A — domain fit matters more than raw benchmark accuracy ✓
 - C) Neither — you need to train your own model
 - D) Both — run them in parallel and average the results
4. What is the correct order of steps when using a pre-trained model?
 - A) Prepare input → Load model → Run inference
 - B) Run inference → Load model → Prepare input
 - C) Load model → Prepare input → Run inference ✓
 - D) Load model → Run inference → Prepare input

5. After calling `model.predict([[5.1, 3.5, 1.4, 0.2]])` on a trained iris classifier, you get `[0]`. What does this mean?
- A) The model failed with an error code 0
 - B) The model predicts class index 0 — which maps to "setosa" ✓
 - C) The model found 0 similar samples in the training data
 - D) The model is 0% confident in its prediction
6. What is the key difference between training a model and running inference?
- A) Training uses more memory than inference
 - B) Inference is done once; training is done many times
 - C) Training updates the model's parameters; inference uses them to make predictions without changing them ✓
 - D) There is no meaningful difference — they are the same process
7. You download a model from Hugging Face and want to run the same model prediction in a different project without downloading it again. What should you do?
- A) Save the model to disk with joblib and load it in the new project ✓
 - B) Copy the Hugging Face URL and paste it directly into your code
 - C) Retrain the model from scratch in the new project
 - D) The model cannot be reused — you must always download it
8. A model card says: "Achieves 99.5% accuracy on the training set." Why is this potentially misleading?
- A) 99.5% accuracy is too high to be real
 - B) Training set accuracy measures memorization, not generalization — the model may overfit ✓
 - C) Accuracy is not a valid metric for classifiers
 - D) The model should be evaluated on a benchmark dataset only

Evaluation Criteria:

- Correctly identifies pre-trained models as the starting point before training from scratch
- Applies license and domain fit criteria correctly
- Understands the load-prepare-predict order
- Distinguishes training from inference

Score Interpretation:

- 8/8: You're ready to integrate pre-trained models into real applications
- 6–7: Review any missed lessons before moving on
- 4–5: Re-read Sections 01 and 02
- Below 4: Restart from Section 01

Content Outline:

- Course recap: what students accomplished
 - Understood why pre-trained models are usually the right starting point
 - Learned where to find models (Hugging Face, scikit-learn, TF Hub, PyTorch Hub)
 - Applied evaluation criteria: task fit, metrics, license, domain, size, community
 - Read and interpreted model cards
 - Ran the load-prepare-predict pipeline in scikit-learn
 - Loaded a classifier, ran inference on new samples, interpreted outputs
- Connection to bigger picture: every ML-powered feature in production started with this question — "does a pre-trained model already exist for this task?" Most of the time, the answer is yes. The skills you've built here — finding, evaluating, and using pre-trained models — are the practical foundation of ML engineering.
- Next steps:
 - The remaining learnpacks cover training models from scratch (when pre-trained options don't exist or don't fit)
 - Fine-tuning: taking a pre-trained model and further training it on your specific domain data — the best of both worlds
 - Deployment: serving a model as a REST API endpoint so other services can call it
- Resources for further exploration:
 - Hugging Face documentation: [transformers](#) library, model hub, pipeline API
 - scikit-learn user guide: supervised learning, model persistence
 - Papers With Code ([paperswithcode.com](#)): state-of-the-art models for each task, with benchmark comparisons
 - Model Cards template (Hugging Face): what a complete model card looks like
- Encouragement: the hardest part of using a pre-trained model is often finding the right one — not the code. The code is 10 lines. The judgment about task fit, license, and domain is what distinguishes a software developer who can effectively use ML from one who can't. You now have that judgment.

Note: This is conclusion only — no theory, exercises, or assessment.